ECA5801-IOE AND AI DRIVEN SYSTEM

SIMULATION

EXPERIMENT 13

K.POOJA SREE

192512245

Implementation and Performance Evaluation of a CNN-Based Deep Learning Model for IoE Edge Applications

```
import os, time, random, zipfile
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms
from sklearn.metrics import confusion_matrix, classification_report

# 1. SETTINGS
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
```

```
try:
    torch.set_num_interop_threads(1)
except RuntimeError:
    pass

torch.set_num_threads(2)
DEVICE = "cpu"
OUT = "/content/experiment5_fast_outputs"
os.makedirs(OUT, exist_ok=True)

EPOCHS = 3
BATCH = 128

print("Downloading/loading MNIST...")

# 2. DATASET
transform = transforms.ToTensor()

train_data = datasets.MNIST(
    "/content/mnist_data", train=True,
    download=True, transform=transform
)
```

```
test_data = datasets.MNIST(
    "/content/mnist_data", train=False,
    download=True, transform=transform
)

# Use a fixed subset for faster training.
# 15,000 training images and 3,000 validation images.
rng = np.random.default_rng(SEED)
train_idx = rng.choice(60000, 15000, replace=False)
remaining = np.setdiff1d(np.arange(60000), train_idx)
val_idx = rng.choice(remaining, 3000, replace=False)

train_loader = DataLoader(
    Subset(train_data, train_idx),
    batch_size=BATCH, shuffle=True, num_workers=0
)
val_loader = DataLoader(
    Subset(train_data, val_idx),
    batch_size=256, shuffle=False, num_workers=0
)
test_loader = DataLoader(
    test_data, batch_size=256,
```

```
    shuffle=False, num_workers=0
)

# 3. LIGHTWEIGHT CNN
class FastCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),       # 28 -> 14

            nn.Conv2d(16, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)        # 14 -> 7
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, 64),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(64, 10)
```

```
        )

    def forward(self, x):
        return self.classifier(self.features(x))

model = FastCNN().to(DEVICE)
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()

params = sum(p.numel() for p in model.parameters())
print("Parameters:", f"{params:,}")

# 4. TRAINING
history = {
    "train_loss": [], "val_loss": [],
    "train_acc": [], "val_acc": []
}

def evaluate(net, loader):
    net.eval()
    loss_sum, correct, total = 0.0, 0, 0
```

```
    with torch.inference_mode():
        for x, y in loader:
            output = net(x)
            loss_sum += criterion(output, y).item() * len(y)
            correct += (output.argmax(1) == y).sum().item()
            total += len(y)

    return loss_sum / total, 100 * correct / total

best_acc = 0
best_state = None

for epoch in range(EPOCHS):
    model.train()
    total_loss, correct, total = 0.0, 0, 0

    start = time.perf_counter()

    for x, y in train_loader:
        optimizer.zero_grad(set_to_none=True)
        output = model(x)
        loss = criterion(output, y)
```

```
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * len(y)
        correct += (output.argmax(1) == y).sum().item()
        total += len(y)

    train_loss = total_loss / total
    train_acc = 100 * correct / total
    val_loss, val_acc = evaluate(model, val_loader)

    history["train_loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    history["train_acc"].append(train_acc)
    history["val_acc"].append(val_acc)

    if val_acc > best_acc:
        best_acc = val_acc
        best_state = {
            k: v.detach().clone()
            for k, v in model.state_dict().items()
        }
```

```
    print(
        f"Epoch {epoch+1}/{EPOCHS} | "
        f"Train loss={train_loss:.4f}, "
        f"Train acc={train_acc:.2f}% | "
        f"Val loss={val_loss:.4f}, "
        f"Val acc={val_acc:.2f}% | "
        f"{time.perf_counter()-start:.1f}s"
    )

model.load_state_dict(best_state)
model.eval()

# 5. TRAINING CURVES
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(history["train_loss"], marker="o", label="Train")
plt.plot(history["val_loss"], marker="o", label="Validation")
plt.title("Loss Curves")
plt.xlabel("Epoch")
plt.ylabel("Loss")
```

```
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(history["train_acc"], marker="o", label="Train")
plt.plot(history["val_acc"], marker="o", label="Validation")
plt.title("Accuracy Curves")
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(f"{OUT}/training_curves.png", dpi=140)
plt.show()

pd.DataFrame(history).to_csv(
    f"{OUT}/training_history.csv", index=False
)

# 6. TEST METRICS
model.eval()
```

```
y_true, y_pred, y_probs = [], [], []
test_loss_sum = 0.0

with torch.inference_mode():
    for x, y in test_loader:
        output = model(x)
        test_loss_sum += criterion(output, y).item() * len(y)
        probs = torch.softmax(output, dim=1)

        y_true.extend(y.numpy().tolist())
        y_pred.extend(probs.argmax(1).numpy().tolist())
        y_probs.extend(probs.numpy().tolist())

test_acc = 100 * np.mean(np.array(y_true) == np.array(y_pred))
test_loss = test_loss_sum / len(test_data)

print("\nTEST ACCURACY:", f"{test_acc:.2f}%")
print("TEST LOSS:", f"{test_loss:.4f}")

report = classification_report(
    y_true, y_pred, output_dict=True, zero_division=0
)
```

```
report_df = pd.DataFrame(report).transpose()

print("\nCLASSIFICATION REPORT")

print(report_df.round(4))

report_df.to_csv(f"{OUT}/classification_report.csv")


# 7. CONFUSION MATRIX

cm = confusion_matrix(y_true, y_pred)


plt.figure(figsize=(8, 7))

plt.imshow(cm, cmap="Blues")

plt.title("MNIST Confusion Matrix")

plt.xlabel("Predicted class")

plt.ylabel("True class")

plt.xticks(range(10))

plt.yticks(range(10))

plt.colorbar()


for i in range(10):

    for j in range(10):

        plt.text(j, i, str(cm[i, j]),

                 ha="center", va="center", fontsize=7)
```

```
plt.tight_layout()
plt.savefig(f"{OUT}/confusion_matrix.png", dpi=140)
plt.show()

# 8. EIGHT SAMPLE PREDICTIONS
sample_ids = rng.choice(len(test_data), 8, replace=False)
fig, axes = plt.subplots(2, 4, figsize=(10, 5))
sample_rows = []

for ax, idx in zip(axes.flat, sample_ids):
    image, label = test_data[int(idx)]

    with torch.inference_mode():
        probabilities = torch.softmax(
            model(image.unsqueeze(0)), dim=1
        )[0]

    pred = int(probabilities.argmax())
    confidence = float(probabilities[pred]) * 100

    ax.imshow(image.squeeze(), cmap="gray")
    ax.set_title(
```

```
        f"True: {label} | Pred: {pred}\n"
        f"Confidence: {confidence:.1f}%",
        color="green" if pred == label else "red",
        fontsize=9
    )
    ax.axis("off")

    sample_rows.append({
        "True label": label,
        "Predicted label": pred,
        "Confidence (%)": round(confidence, 2),
        "Correct": pred == label
    })

plt.tight_layout()
plt.savefig(f"{OUT}/sample_predictions.png", dpi=140)
plt.show()

sample_df = pd.DataFrame(sample_rows)
print("\nEIGHT SAMPLE PREDICTIONS")
print(sample_df.to_string(index=False))
sample_df.to_csv(f"{OUT}/sample_predictions.csv", index=False)
```

```
# 9. SAVE MODEL AND SIZE
model_path = f"{OUT}/fast_cnn_mnist.pth"
torch.save(model.state_dict(), model_path)

size_mb = os.path.getsize(model_path) / 1_000_000
parameter_mb = params * 4 / 1_000_000

# 10. FLOPs ESTIMATE: Conv2d and Linear layers
macs = 0
def count_ops(layer, inputs, output):
    global macs
    if isinstance(layer, nn.Conv2d):
        kh, kw = layer.kernel_size
        macs += (
            output.shape[0] * output.shape[2] * output.shape[3]
            * layer.out_channels * kh * kw
            * (layer.in_channels // layer.groups)
        )
    elif isinstance(layer, nn.Linear):
        macs += inputs[0].shape[0] * layer.in_features * layer.out_features
```

```
hooks = []
for layer in model.modules():
    if isinstance(layer, (nn.Conv2d, nn.Linear)):
        hooks.append(layer.register_forward_hook(count_ops))

with torch.inference_mode():
    model(torch.zeros(1, 1, 28, 28))

for h in hooks:
    h.remove()

flops = 2 * macs

# 11. LATENCY: 10 warmups, 100 timed runs
example = test_data[0][0].unsqueeze(0)

with torch.inference_mode():
    for _ in range(10):
        model(example)

latencies = []
```

```
with torch.inference_mode():
    for _ in range(100):
        t0 = time.perf_counter()
        model(example)
        latencies.append((time.perf_counter() - t0) * 1000)

mean_ms = float(np.mean(latencies))
std_ms = float(np.std(latencies, ddof=1))
p95_ms = float(np.percentile(latencies, 95))
fps = 1000 / mean_ms

plt.figure(figsize=(7, 4))
plt.hist(latencies, bins=12, edgecolor="black")
plt.axvline(mean_ms, linestyle="--", label=f"Mean={mean_ms:.2f} ms")
plt.axvline(p95_ms, linestyle=":", label=f"P95={p95_ms:.2f} ms")
plt.xlabel("Latency (ms)")
plt.ylabel("Frequency")
plt.title("CPU Inference Latency")
plt.legend()
plt.tight_layout()
plt.savefig(f"{OUT}/latency.png", dpi=140)
plt.show()
```

```
# 12. RESOURCE SNAPSHOT
try:
    import psutil
    rss_mb = psutil.Process(os.getpid()).memory_info().rss / (1024**2)
except Exception:
    rss_mb = float("nan")

# 13. RESULTS TABLE
accuracy_pass = test_acc >= 90
latency_pass = mean_ms <= 50
size_pass = size_mb < 5

results = pd.DataFrame([
    ["Test accuracy (%)", ">= 90%", f"{test_acc:.2f}%"],
    ["Test loss", "Record measured value", f"{test_loss:.4f}"],
    ["Mean latency", "<= 50 ms", f"{mean_ms:.3f} ms"],
    ["Latency standard deviation", "Record measured value",
f"{std_ms:.3f} ms"],
    ["P95 latency", "Record measured value", f"{p95_ms:.3f} ms"],
    ["Throughput", "1000 / mean latency", f"{fps:.2f} images/s"],
    ["Saved model size", "< 5 MB", f"{size_mb:.3f} MB"],
```

```
    ["Parameter count", "< 1 million", f"{params:,}"],
    ["FP32 parameter-only size", "Parameters x 4 / 1e6",
f"{parameter_mb:.3f} MB"],
    ["Approximate FLOPs/image", "2 x MACs", f"{flops:,}"],
    ["Process memory snapshot", "Record measured value", f"{rss_mb:.2f}
MB"],
    ["Accuracy threshold", ">= 90%", "PASS" if accuracy_pass else "NOT
MET"],
    ["Latency threshold", "<= 50 ms", "PASS" if latency_pass else "NOT
MET"],
    ["Size threshold", "< 5 MB", "PASS" if size_pass else "NOT MET"],
    ["Overall edge suitability", "Meet all three thresholds",
     "PASS" if accuracy_pass and latency_pass and size_pass else "NEEDS
IMPROVEMENT"]
], columns=["Output parameter", "Expected/target", "Practical value"])

print("\nFINAL OUTPUT TABLE")
print(results.to_string(index=False))
results.to_csv(f"{OUT}/output_parameter_table.csv", index=False)

summary = f"""
EXPERIMENT 5 — FAST MNIST CNN
```

Dataset: MNIST (28x28 grayscale, 10 classes)

Epochs: {EPOCHS}

Training subset: {len(train_idx)} images

Validation subset: {len(val_idx)} images

Optimizer: Adam

Learning rate: 0.001

CPU threads: {torch.get_num_threads()}

Test accuracy: {test_acc:.2f}%

Test loss: {test_loss:.4f}

Parameters: {params:,}

Approximate FLOPs/image: {flops:,}

Serialized model size: {size_mb:.3f} MB

Parameter-only FP32 estimate: {parameter_mb:.3f} MB

Mean latency: {mean_ms:.3f} ms

Latency standard deviation: {std_ms:.3f} ms

P95 latency: {p95_ms:.3f} ms

Throughput: {fps:.2f} images/s

Process memory snapshot: {rss_mb:.2f} MB

Accuracy >= 90%: {"PASS" if accuracy_pass else "NOT MET"}

Latency <= 50 ms: {"PASS" if latency_pass else "NOT MET"}

Model size < 5 MB: {"PASS" if size_pass else "NOT MET"}

Memory note: Process memory is a whole-process snapshot, not isolated peak inference RAM. FLOPs are an estimate for Conv2D and Linear layers.

The model was trained on a subset for speed, so this is a fast simulation rather than the full 15-epoch CIFAR-10 experiment.

"""

```
with open(f"{OUT}/experiment_summary.txt", "w") as f:
    f.write(summary)

# 14. ZIP ALL OUTPUTS
zip_path = "/content/experiment5_fast_results.zip"

with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as z:
    for filename in os.listdir(OUT):
        z.write(f"{OUT}/{filename}", arcname=filename)

print("\n" + summary)
print("Results saved to:", OUT)
print("ZIP:", zip_path)
```

try:

from google.colab import files

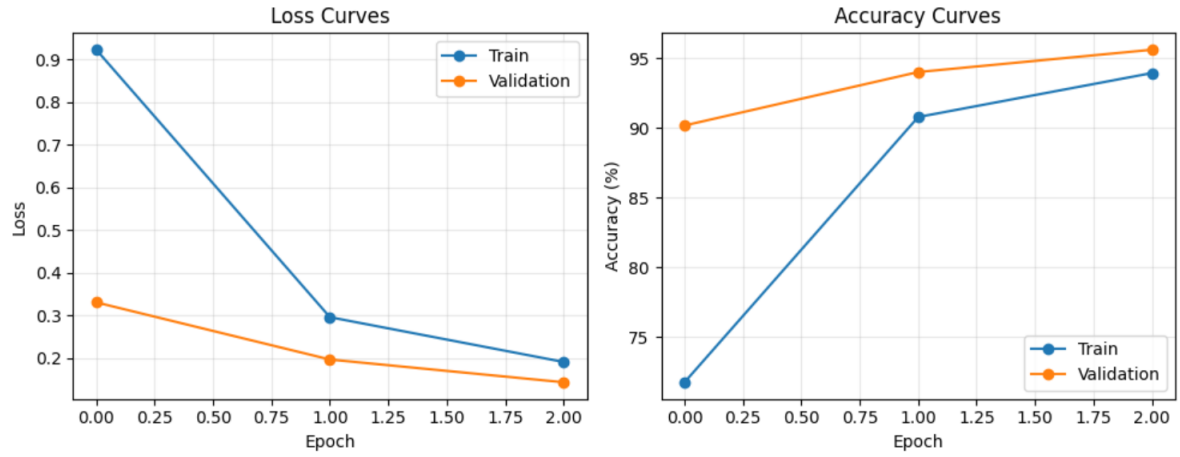
files.download(zip_path)

except Exception:

print("Download experiment5_fast_results.zip from the Colab Files panel.")

print("\nBROOO, FAST EXPERIMENT COMPLETE!")

```
Downloading/loading MNIST...
100%|██████████| 9.91M/9.91M [00:00<00:00, 63.2MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 1.73MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 15.2MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 8.96MB/s]
Parameters: 105,866
Epoch 1/3 | Train loss=0.9229, Train acc=71.75% | Val loss=0.3309, Val acc=90.17% | 12.2s
Epoch 2/3 | Train loss=0.2962, Train acc=90.77% | Val loss=0.1970, Val acc=94.00% | 13.1s
Epoch 3/3 | Train loss=0.1914, Train acc=93.93% | Val loss=0.1438, Val acc=95.60% | 12.5s
```



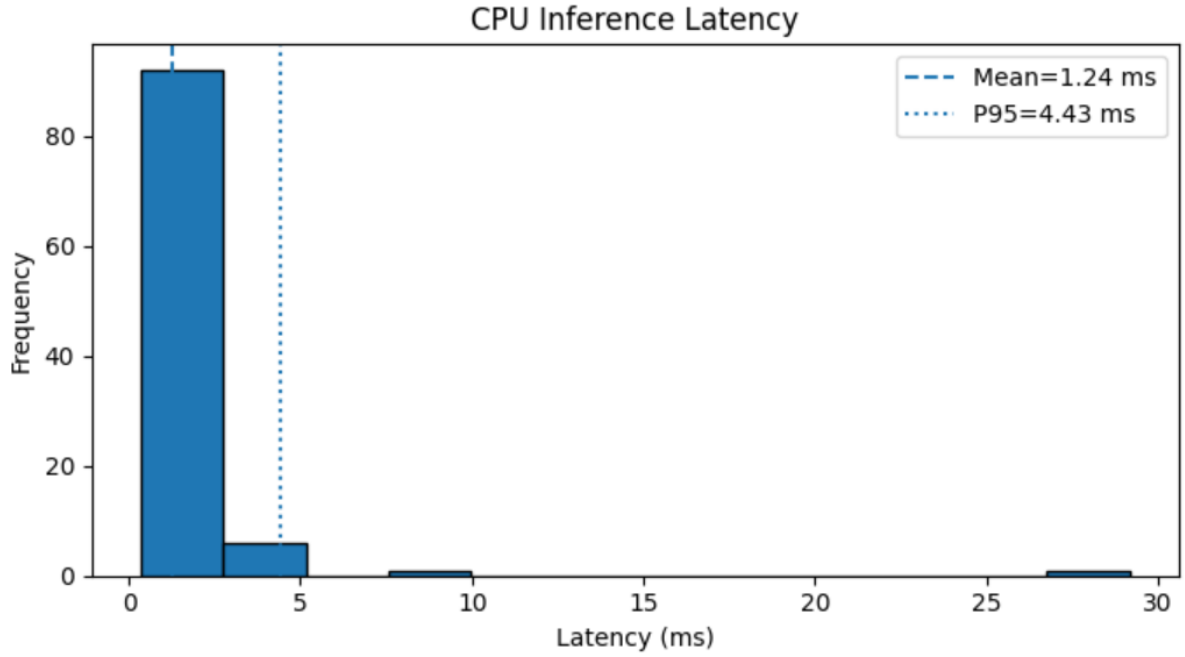

```
TEST ACCURACY: 96.05%
TEST LOSS: 0.1229
```

CLASSIFICATION REPORT

	precision	recall	f1-score	support
0	0.9614	0.9908	0.9759	980.0000
1	0.9867	0.9806	0.9837	1135.0000
2	0.9647	0.9535	0.9591	1032.0000
3	0.9457	0.9663	0.9559	1010.0000
4	0.9842	0.9491	0.9663	982.0000
5	0.9595	0.9563	0.9579	892.0000
6	0.9786	0.9562	0.9673	958.0000
7	0.9644	0.9475	0.9558	1028.0000
8	0.9286	0.9476	0.9380	974.0000
9	0.9313	0.9544	0.9427	1009.0000
accuracy	0.9605	0.9605	0.9605	0.9605
macro avg	0.9605	0.9602	0.9603	10000.0000
weighted avg	0.9608	0.9605	0.9605	10000.0000

EIGHT SAMPLE PREDICTIONS

True label	Predicted label	Confidence (%)	Correct
8	8	82.54	True
0	0	99.99	True
3	3	99.90	True
0	0	99.82	True
2	2	98.94	True
8	8	98.97	True
8	8	85.27	True
5	5	89.35	True



FINAL OUTPUT TABLE

Output parameter	Expected/target	Practical value
Test accuracy (%)	>= 90%	96.05%
Test loss	Record measured value	0.1229
Mean latency	<= 50 ms	1.240 ms
Latency standard deviation	Record measured value	3.114 ms
P95 latency	Record measured value	4.430 ms
Throughput	1000 / mean latency	806.24 images/s
Saved model size	< 5 MB	0.427 MB
Parameter count	< 1 million	105,866
FP32 parameter-only size	Parameters x 4 / 1e6	0.423 MB
Approximate FLOPs/image	2 x MACs	2,234,112
Process memory snapshot	Record measured value	621.20 MB
Accuracy threshold	>= 90%	PASS
Latency threshold	<= 50 ms	PASS
Size threshold	< 5 MB	PASS
Overall edge suitability	Meet all three thresholds	PASS

EXPERIMENT 5 — FAST MNIST CNN

Dataset: MNIST (28x28 grayscale, 10 classes)
Epochs: 3
Training subset: 15000 images
Validation subset: 3000 images
Optimizer: Adam
Learning rate: 0.001
CPU threads: 2

Test accuracy: 96.05%
Test loss: 0.1229
Parameters: 105,866
Approximate FLOPs/image: 2,234,112
Serialized model size: 0.427 MB
Parameter-only FP32 estimate: 0.423 MB
Mean latency: 1.240 ms
Latency standard deviation: 3.114 ms
P95 latency: 4.430 ms
Throughput: 806.24 images/s
Process memory snapshot: 621.20 MB

Accuracy >= 90%: PASS
Latency <= 50 ms: PASS
Model size < 5 MB: PASS

Memory note: Process memory is a whole-process snapshot, not isolated
peak inference RAM. FLOPs are an estimate for Conv2D and Linear layers.
The model was trained on a subset for speed, so this is a fast simulation
rather than the full 15-epoch CIFAR-10 experiment.

Results saved to: /content/experiment5_fast_outputs
ZIP: /content/experiment5_fast_results.zip

BROOO, FAST EXPERIMENT COMPLETE!